# COGNIZANT QEA INTERIM ROUND

## Focused Five-Day Preparation Guide

**Java | SQL | Manual Testing | Agile | QEA and HR**

*Simple interview answers, explained Java keywords, basic programs, one-section SQL revision, testing concepts, and role-focused professional questions.*

# How to Use This Guide

Prepare the concepts in the order shown. Read an answer, close the guide, and explain the same concept in your own words. Most interview answers should take 30 to 75 seconds. For Java programs and SQL queries, write the code on a blank page before checking the answer.

- Definition first: state what the term means.
- Purpose second: explain why it is used.
- Example third: give one small practical example.
- Difference last: when two terms are similar, explain the comparison clearly.
- Never claim project experience you do not have. Use training, academic, or practice examples honestly.

# Five-Day Schedule

| Day | Focus | Topics | Practice output |
|---|---|---|---|
| 1 | Java basics | Java, JDK/JRE/JVM, class, object, constructor, OOP pillars, keywords | Explain every keyword aloud and write two class examples |
| 2 | Java applied | Exceptions, collections, strings, arrays, basic programs | Write palindrome, factorial, prime, Armstrong, Fibonacci, reverse |
| 3 | SQL | Database basics, CRUD, commands, keys, filters, joins, grouping, queries | Write all sample queries without copying |
| 4 | Manual testing | Core terms, techniques, SDLC/STLC, defects, test artifacts, Agile | Build one login test case and explain one defect lifecycle |
| 5 | QEA and HR | Project approach, starting testing, new build, priorities, self-introduction | Two spoken mock interviews and final revision |

# DAY 1 - JAVA FUNDAMENTALS AND OOP

## 1. What is Java?

**Interview answer:** Java is a high-level, class-based, object-oriented programming language. Java source code is compiled into bytecode, and the Java Virtual Machine executes that bytecode. Because suitable JVM implementations are available for different operating systems, the same compiled Java program can run across platforms.

- Important features include platform independence, object orientation, automatic memory management, exception handling, multithreading, portability, and a large standard library.

**Remember:** Java follows the idea “write once, run anywhere” through bytecode and the JVM.

## 2. Explain JDK, JRE, and JVM.

**Interview answer:** JDK is the development kit used to create and compile Java programs. JRE provides the libraries and runtime environment needed to run a Java application. JVM is the virtual machine that loads, verifies, and executes Java bytecode.

- JDK contains development tools such as the compiler.
- JRE contains runtime libraries and the JVM.
- JVM executes bytecode and manages runtime activities such as memory.

**Remember:** JDK develops, JRE provides the runtime, and JVM executes bytecode.

## 3. What is a class?

**Interview answer:** A class is a blueprint that defines the data and behavior of objects. It can contain fields, methods, constructors, blocks, and nested types.

```
class Employee {
    int id;
    String name;

    void display() {
        System.out.println(id + " " + name);
    }
}
```

## 4. What is an object?

**Interview answer:** An object is a runtime instance of a class. It has state, behavior, and identity. The new keyword creates an object, while a reference variable stores the reference used to access that object.

```
Employee employee = new Employee();
employee.id = 101;
employee.name = "Lakshay";
employee.display();
```

**Remember:** A class is the blueprint; an object is an instance created from it.

## 5. What is a constructor?

**Interview answer:** A constructor initializes a new object. It has the same name as the class and no return type. Constructors can be no-argument or parameterized, and they can be overloaded.

```
class Employee {
   int id;
   String name;

   Employee(int id, String name) {
      this.id = id;
      this.name = name;
   }
}
```

## 6. What is Object-Oriented Programming?

**Interview answer:** Object-Oriented Programming, or OOP, organizes software around classes and objects. It combines data and behavior and helps create modular, reusable, maintainable, and extensible programs.

- The four main pillars are encapsulation, inheritance, polymorphism, and abstraction.

## 7. What is encapsulation?

**Interview answer:** Encapsulation binds data and related methods inside a class and controls access to the internal state. In Java, fields are commonly marked private and accessed through methods that can validate changes.

```
class Account {
   private double balance;

   public void deposit(double amount) {
      if (amount > 0) balance += amount;
   }

   public double getBalance() { return balance; }
}
```

**Remember:** Encapsulation provides data hiding and controlled access.

## 8. What is inheritance?

**Interview answer:** Inheritance allows a child class to acquire accessible fields and methods from a parent class and add or change behavior. It represents an is-a relationship and supports reuse and runtime polymorphism.

```
class Animal {
   void eat() { System.out.println("Eating"); }
}
class Dog extends Animal {
   void bark() { System.out.println("Barking"); }
}
```

**Remember:** Use inheritance when the child is genuinely a specialized form of the parent.

### 9. What is polymorphism?

**Interview answer:** Polymorphism means one name, reference, or contract can represent multiple forms. Compile-time polymorphism is commonly achieved through method overloading. Runtime polymorphism is achieved when a child class overrides an inherited method.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
}

class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); }
}
```

**Remember:** Overloading changes parameters; overriding changes inherited behavior.

### 10. What is abstraction?

**Interview answer:** Abstraction exposes essential operations while hiding unnecessary implementation details. Java supports abstraction through abstract classes and interfaces.

```
interface Payment {
    void pay(double amount);
}
class CardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid: " + amount);
    }
}
```

**Remember:** Abstraction explains what an object can do without exposing every implementation detail.

# Java Keywords Used in OOP

*These are the keywords interviewers commonly ask after the four pillars.*

| Keyword or term | Meaning | Example |
|---|---|---|
| class | Declares a class, which acts as a blueprint for objects. | class Employee { } |
| new | Creates a new object and invokes its constructor. | Employee e = new Employee(); |
| this | Refers to the current object. It is often used to distinguish an instance field from a parameter with the same name or to call another constructor in the same class. | this.name = name;  // current object field |
| super | Refers to the immediate parent-class part of the current object. It can access a parent method or constructor. super() must be the first explicit statement in a constructor. | super();  or  super.display(); |
| extends | Creates inheritance between classes or allows one interface to inherit another interface. A Java class can directly extend only one class. | class Dog extends Animal { } |

| | | |
|---|---|---|
| implements | Declares that a class follows one or more interface contracts. The class must implement required abstract methods unless the class is abstract. | class CardPayment implements Payment { } |
| interface | Declares a contract that implementing classes follow. Interfaces support abstraction and multiple type inheritance. | interface Payment { void pay(); } |
| abstract | Marks an incomplete class or a method without an implementation. An abstract class cannot be instantiated directly. | abstract class Vehicle { abstract void start(); } |
| private | Allows access only within the declaring class. Commonly used for data hiding. | private double balance; |
| default access | When no access modifier is written, access is limited to the same package. Default access is not itself a Java keyword. | int count; |
| protected | Allows access in the same package and in subclasses according to Java access rules. | protected void display() { } |
| public | Allows access from any class when the containing type is accessible. | public void login() { } |
| static | Makes a member belong to the class rather than to each object. A static method cannot directly use instance members without an object. | static int count; |
| final | A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended. | final double TAX = 0.18; |
| void | Indicates that a method does not return a value. | void display() { } |
| return | Ends a method and optionally sends a value back to the caller. | return total; |
| package | Places related classes and interfaces in a named package and helps organize code and control access. | package com.example.qea; |
| import | Makes a type from another package available by its simple name. | import java.util.ArrayList; |
| instanceof | Checks whether an object reference is compatible with a particular type. | if (obj instanceof Employee) { } |
| @Override | This is an annotation, not a keyword. It asks the compiler to confirm that a method correctly overrides an inherited method. | @Override public String toString() { ... } |

# DAY 2 - EXCEPTIONS, COLLECTIONS, AND PROGRAMS

## 11. What is exception handling?

**Interview answer:** Exception handling is a structured mechanism for handling abnormal situations during program execution. It allows a program to report or recover from a problem instead of terminating without control.

- Examples include invalid numeric input, division by zero, a missing file, an invalid array index, and use of a null reference.

## 12. Explain try, catch, finally, throw, and throws.

**Interview answer:** The try block contains code that may fail. A catch block handles a matching exception. The finally block normally runs whether an exception occurs or not and is commonly used for cleanup. throw explicitly creates or passes an exception. throws declares that a method may pass specific exceptions to its caller.

```
try {
    int value = Integer.parseInt(input);
} catch (NumberFormatException ex) {
    System.out.println("Enter a valid number");
} finally {
    System.out.println("Completed");
}

void readFile() throws IOException {
    throw new IOException("File unavailable");
}
```

**Remember:** throw is used inside code to throw an exception; throws is used in a method declaration.

## 13. What are checked and unchecked exceptions?

**Interview answer:** Checked exceptions are checked by the compiler and must be handled or declared, for example IOException and SQLException. Unchecked exceptions are RuntimeException subclasses and commonly represent programming or validation problems, for example NullPointerException, ArithmeticException, and NumberFormatException.

## 14. What is the Java Collections Framework?

**Interview answer:** The Java Collections Framework provides interfaces and classes for storing and manipulating groups of objects. The main collection interfaces are List, Set, and Queue. Map belongs to the framework but stores key-value pairs rather than extending Collection.

## 15. Explain List, Set, Queue, and Map.

**Interview answer:** A List is ordered and allows duplicates. A Set stores unique elements. A Queue generally processes elements in an order such as first-in, first-out. A Map stores key-value pairs with unique keys.

- ArrayList provides fast indexed access and is a common List implementation.
- LinkedList supports list and deque operations.
- HashSet stores unique values without a guaranteed iteration order.
- LinkedHashSet preserves insertion order.
- TreeSet stores values in sorted order.
- HashMap stores key-value pairs without guaranteed order.
- LinkedHashMap preserves insertion order.
- TreeMap sorts entries by key.

```
List<String> names = new ArrayList<>();
names.add("Lakshay");
names.add("Lakshay");

Set<Integer> ids = new HashSet<>();
ids.add(101);
ids.add(101); // duplicate not added

Map<Integer, String> people = new HashMap<>();
people.put(101, "Lakshay");
```

### 16. ArrayList versus LinkedList.

**Interview answer:** ArrayList uses a resizable array and is usually better for indexed access. LinkedList stores linked nodes and can be useful for frequent insertion or removal through an iterator or at the ends. The best choice depends on the operations, not only the collection size.

# Basic Java Programs

### 17. Palindrome string

**Interview answer:** A palindrome reads the same forward and backward.

```
String text = "madam";
String reversed = new StringBuilder(text).reverse().toString();
System.out.println(text.equalsIgnoreCase(reversed)
        ? "Palindrome" : "Not palindrome");
```

### 18. Palindrome number

**Interview answer:** Reverse the number digit by digit, then compare it with the original value.

```
int number = 121, original = number, reversed = 0;
while (number > 0) {
    int digit = number % 10;
    reversed = reversed * 10 + digit;
    number /= 10;
}
System.out.println(original == reversed ? "Palindrome" : "Not palindrome");
```

### 19. Factorial

**Interview answer:** The factorial of n is the product of all positive integers from 1 to n. Zero factorial is 1.

```
int n = 5;
long factorial = 1;
for (int i = 2; i <= n; i++) factorial *= i;
System.out.println(factorial);
```

### 20. Prime number

**Interview answer:** A prime number is greater than 1 and has exactly two positive factors. Test divisors only up to the square root.

```
int n = 29;
boolean prime = n >= 2;
for (int i = 2; i * i <= n && prime; i++) {
    if (n % i == 0) prime = false;
}
System.out.println(prime ? "Prime" : "Not prime");
```

## 21. Armstrong number

**Interview answer:** An Armstrong number equals the sum of each digit raised to the number of digits. For 153, 1 cubed + 5 cubed + 3 cubed equals 153.

```
int n = 153, original = n, sum = 0;
int digits = String.valueOf(n).length();
while (n > 0) {
    int digit = n % 10;
    sum += (int) Math.pow(digit, digits);
    n /= 10;
}
System.out.println(sum == original ? "Armstrong" : "Not Armstrong");
```

## 22. Fibonacci series

**Interview answer:** Each term is the sum of the previous two terms.

```
int first = 0, second = 1;
for (int i = 0; i < 10; i++) {
    System.out.print(first + " ");
    int next = first + second;
    first = second;
    second = next;
}
```

## 23. Reverse a number

**Interview answer:** Take the last digit using remainder, append it to the reversed value, and remove it using integer division.

```
int n = 1234, reversed = 0;
while (n > 0) {
    reversed = reversed * 10 + n % 10;
    n /= 10;
}
System.out.println(reversed);
```

## 24. Largest of three numbers

**Interview answer:** Start with one value as the largest and compare the remaining values.

```
int a = 10, b = 25, c = 15;
int largest = a;
if (b > largest) largest = b;
if (c > largest) largest = c;
System.out.println(largest);
```

# DAY 3 - SQL IN TWO-PAGE REVISION FORMAT

## 25. What is SQL?

**Interview answer:** SQL stands for Structured Query Language. It is used to define database structures and to create, read, update, delete, filter, group, join, and control relational data.

## 26. What are database, table, row, column, and NULL?

**Interview answer:** A database is an organized collection of data. A table stores related data in rows and columns. A row represents one record, a column represents one attribute, and NULL represents a missing or unknown value.

```
SELECT * FROM Employee WHERE department_id IS NULL;
```

## 27. What are SQL command categories?

**Interview answer:** DDL defines structures: CREATE, ALTER, DROP, and TRUNCATE. DML changes data: INSERT, UPDATE, and DELETE. DQL retrieves data using SELECT. TCL controls transactions: COMMIT, ROLLBACK, and SAVEPOINT. DCL controls permissions: GRANT and REVOKE.

## 28. Explain CRUD operations.

**Interview answer:** CRUD means Create, Read, Update, and Delete. In SQL these operations are commonly performed using INSERT, SELECT, UPDATE, and DELETE.

```
INSERT INTO Employee (employee_id, employee_name, salary)
VALUES (101, 'Lakshay', 40000);

SELECT * FROM Employee;

UPDATE Employee SET salary = 45000
WHERE employee_id = 101;

DELETE FROM Employee
WHERE employee_id = 101;
```

**Remember:** Use WHERE carefully with UPDATE and DELETE, otherwise many rows may be changed.

## 29. Explain Primary Key and Foreign Key.

**Interview answer:** A Primary Key uniquely identifies each row and cannot contain duplicate or null values. A Foreign Key references a key in another or the same table and helps enforce relationships between records.

```
CREATE TABLE Department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50)
);

CREATE TABLE Employee (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(50),
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES Department(department_id)
);
```

## 30. DELETE versus TRUNCATE versus DROP.

**Interview answer:** DELETE removes selected rows and supports WHERE. TRUNCATE removes all rows while keeping the table structure. DROP removes the table object itself. Transaction and rollback behavior for TRUNCATE and DROP depends on the database system.

```
DELETE FROM Employee WHERE employee_id = 101;
TRUNCATE TABLE Employee_Stage;
DROP TABLE Employee_Archive;
```

## 31. WHERE, ORDER BY, DISTINCT, and aliases.

**Interview answer:** WHERE filters rows. ORDER BY sorts results. DISTINCT removes duplicate result rows. An alias gives a temporary readable name to a table or output column.

```
SELECT DISTINCT e.department_id, e.salary AS monthly_salary
FROM Employee e
WHERE e.salary >= 40000
ORDER BY e.salary DESC;
```

## 32. Aggregate functions, GROUP BY, and HAVING.

**Interview answer:** COUNT, SUM, AVG, MIN, and MAX summarize data. GROUP BY forms groups of matching values. HAVING filters grouped results, while WHERE filters rows before grouping.

```
SELECT department_id, COUNT(*) AS employees, AVG(salary) AS average_salary
FROM Employee
WHERE status = 'ACTIVE'
GROUP BY department_id
HAVING COUNT(*) > 5;
```

## 33. What are SQL joins?

**Interview answer:** A join combines related rows from table sources. INNER JOIN returns matches. LEFT JOIN keeps every left row. RIGHT JOIN keeps every right row. FULL OUTER JOIN keeps matching and non-matching rows from both sides. CROSS JOIN returns every combination.

```
SELECT e.employee_name, d.department_name
FROM Employee e
INNER JOIN Department d
    ON e.department_id = d.department_id;
```

## 34. Useful SQL interview queries.

**Interview answer:** Practise these short queries until you can write them without copying.

```
-- Second-highest distinct salary
SELECT MAX(salary) AS second_highest_salary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Duplicate names
SELECT employee_name, COUNT(*) AS occurrences
FROM Employee
GROUP BY employee_name
HAVING COUNT(*) > 1;

-- Employees above average salary
SELECT * FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);

-- Records with no department
SELECT * FROM Employee
WHERE department_id IS NULL;
```

# DAY 4 - MANUAL TESTING AND AGILE

## 35. What is software testing?

**Interview answer:** Software testing is the planned process of evaluating software and related work products to identify defects, verify requirements, and provide information about quality and risk.

- Testing includes reviews, planning, test design, data preparation, execution, defect reporting, retesting, regression, and closure.

**Remember:** Testing shows the presence of defects and provides confidence; it cannot prove that no defect exists.

## 36. Error, defect, bug, and failure.

**Interview answer:** An error is a human mistake. A defect is a flaw introduced into a requirement, design, code, or other work product. Bug is a commonly used informal term for a software defect. A failure is incorrect observable behavior when a defect is executed.

**Remember:** A human error can introduce a defect, and execution of that defect may produce a failure.

## 37. Functional and non-functional testing.

**Interview answer:** Functional testing checks what the system does against business rules and user requirements. Non-functional testing checks how well the system operates through characteristics such as performance, security, usability, compatibility, reliability, accessibility, and recovery.

- Functional example: verify that payment completes with valid data.
- Non-functional example: verify that payment completes within the required response time under expected load.

**Remember:** Functional testing checks what the system does; non-functional testing checks how well it does it.

## 38. What are major testing levels?

**Interview answer:** Unit testing checks individual components. Integration testing checks interfaces and data flow between components. System testing evaluates the complete application. Acceptance testing checks whether the system is suitable for business use.

## 39. Explain common black-box testing techniques.

**Interview answer:** Equivalence Partitioning divides input values into groups expected to behave similarly. Boundary Value Analysis tests values at and around limits. Decision Table Testing covers combinations of conditions and outcomes. State Transition Testing checks behavior when the system moves between states. Error Guessing uses experience to target likely failures.

- For an allowed age of 18 to 60, equivalence partitions are below 18, 18 to 60, and above 60.
- Boundary values can include 17, 18, 19, 59, 60, and 61.

## 40. Severity and priority.

**Interview answer:** Severity represents the impact of a defect on the application or business. Priority represents the urgency with which it should be fixed.

- High severity and high priority: payment fails for all customers.
- High severity and lower priority: a crash in an unused internal feature.
- Low severity and high priority: an incorrect company name on the launch page.
- Low severity and low priority: a minor alignment issue on a rarely used page.

**Remember:** Severity is impact; priority is urgency.

## 41. Explain the defect life cycle.

**Interview answer:** A typical flow is New, Assigned, Open or In Progress, Fixed, Retest, and Closed. If the fix fails, the defect is Reopened. Other possible statuses include Rejected, Duplicate, Deferred, Cannot Reproduce, and Not a Defect.

**Remember:** Close a defect only after verifying the fix in the correct build and checking relevant regression impact.

## 42. Project versus product.

**Interview answer:** A project is temporary work with defined scope, time, and delivery objectives. A product has an ongoing life cycle and is continuously developed for users or customers. A client-specific portal delivery can be a project; a continuously maintained software platform is a product.

## 43. Explain SDLC.

**Interview answer:** SDLC is the Software Development Life Cycle. Typical phases are requirements, planning, design, development, testing, deployment, and maintenance. Testing contributes from the requirement phase through production support.

**Remember:** Testing should start with requirement analysis, not only after coding is complete.

## 44. Explain STLC.

**Interview answer:** STLC is the Software Testing Life Cycle. Its activities include requirement analysis, test planning, test design, test-data preparation, environment setup, execution, defect management, reporting, and closure.

## 45. Verification and validation.

**Interview answer:** Verification checks whether work products are being produced correctly according to specifications and commonly includes reviews and inspections. Validation checks whether the working product satisfies user needs and normally involves executing the software.

- Verification: Are we building the product correctly?
- Validation: Are we building the right product?

## 46. Smoke testing and sanity testing.

**Interview answer:** Smoke testing is a broad, shallow check of critical functions to decide whether a new build is stable for detailed testing. Sanity testing is a focused check of a limited change or defect fix and its immediate impact.

**Remember:** Smoke checks the build; sanity checks the change.

## 47. Retesting and regression testing.

**Interview answer:** Retesting executes the previously failed condition to confirm the fix. Regression testing checks related and existing functionality to detect side effects introduced by the change.

**Remember:** Retesting confirms the fix; regression looks for unintended impact.

## 48. What is a test plan?

**Interview answer:** A test plan explains how testing will be managed. It commonly includes objectives, scope, approach, resources, responsibilities, schedule, environments, tools, data, entry criteria, exit criteria, risks, defect process, reporting, and deliverables.

**Remember:** A test plan explains what, how, who, when, and under what completion criteria.

## 49. What is a test case?

**Interview answer:** A test case is a documented set of preconditions, data, steps, and expected results used to verify specific behavior. Typical fields include ID, title, requirement reference, priority, preconditions, data, steps, expected result, actual result, status, environment, and comments.

TC_LOGIN_001 - Valid login
Precondition: Active user exists
Data: Valid username and password
Steps: Open page; enter credentials; select Login
Expected: User is authenticated and authorized home page appears

## 50. What is Agile?

**Interview answer:** Agile is an iterative and incremental approach in which teams deliver usable value in short cycles, receive feedback, and adapt. Testing is continuous within each iteration rather than being delayed until the end.

## 51. Explain Scrum terminology.

**Interview answer:** The Product Owner maximizes value and manages the Product Backlog. The Scrum Master supports Scrum and team effectiveness. Developers create the usable Increment. A Sprint is a fixed-length cycle. The Product Backlog is the ordered product work. The Sprint Backlog contains the Sprint Goal, selected work, and the plan. A user story describes desired value; acceptance criteria define testable expectations; Definition of Done defines the agreed quality standard for completed work.

## 52. Explain the Agile testing process.

**Interview answer:** The tester understands the user story, reviews acceptance criteria, raises questions, identifies risks, estimates work, prepares scenarios, cases and data, tests during the Sprint, reports defects, retests fixes, runs relevant regression, and communicates quality status. The tester also participates in planning, refinement, Daily Scrum, review, and retrospective.

**Remember:** In Agile, testing and feedback happen continuously throughout the Sprint.

# DAY 5 - QEA AND HR INTERVIEW PREPARATION

## 53. Tell me about yourself.

Good morning. My name is Lakshay Gupta, and I am currently working as an Analyst Trainee at Cognizant in Chennai. I have learned the fundamentals of Manual Testing, Java, and SQL. I am currently learning and practising Selenium. I believe that the best way to learn a technical skill is to understand the concepts and apply them through regular practice. I am interested in QEA, which stands for Quality Engineering and Assurance, because I enjoy analysing applications, identifying defects, and ensuring that software works correctly. I am a quick learner and a dedicated person. My goal is to improve my testing and automation skills and become a successful Quality Engineer.

## 54. What do you know about QEA?

**Interview answer:** QEA stands for Quality Engineering and Assurance. It focuses on quality throughout the development life cycle through requirement review, defect prevention, test planning, functional and non-functional testing, data validation, automation, continuous feedback, and quality reporting. QEA is broader than testing only after development because it aims to identify risks early and improve the delivery process.

**Remember:** QEA combines prevention, testing, automation, and continuous quality improvement.

## 55. Why are you interested in QEA?

**Interview answer:** I am interested in QEA because it combines analytical thinking, technical understanding, and business knowledge. I enjoy understanding how a system should behave, identifying gaps, and verifying results with evidence. QEA also allows me to use manual testing, SQL, Java, Agile practices, and automation together. I want to build strong testing fundamentals and gradually grow into Selenium, API testing, continuous integration, and broader quality engineering.

## 56. How do you approach a new project?

**Interview answer:** I begin by understanding the business objective, users, application scope, and expected outcomes. I review requirements, user stories, acceptance criteria, designs, workflows, integrations, and data flow. I identify high-risk areas and clarify missing information. Then I plan the required testing, prepare scenarios, cases and test data, confirm environment readiness, execute tests, report defects with evidence, retest fixes, perform regression, and communicate coverage and remaining risk.

**Remember:** Understand, clarify, assess risk, plan, design, prepare, execute, report, retest, regress, communicate.

## 57. From where should the testing phase start?

**Interview answer:** Testing should start from requirement analysis. A tester can review requirements, identify ambiguity, check testability, clarify acceptance criteria, identify risks, and begin traceability and test planning before the application is ready.

**Remember:** Testing starts with requirements, not with the completed application.

## 58. What will you do after receiving a new build?

**Interview answer:** I will verify the build number, release notes, deployed environment, included changes, known issues, services, and test data. I will perform smoke testing on critical functions such as application launch, login, navigation, and the main business flow. If the build is stable, I will continue detailed testing. If a critical smoke test fails, I will report it with evidence and communicate that the build may not be suitable for further testing.

## 59. How will you test a new requirement?

**Interview answer:** I will understand the business purpose, review acceptance criteria, and clarify missing rules. Then I will identify positive, negative, boundary, permission, integration, and error-handling conditions. I will prepare cases and data, execute in the correct environment, compare actual and expected results, record evidence, report defects, retest fixes, run relevant regression, and update coverage.

## 60. What will you do if a requirement is unclear?

**Interview answer:** I will identify the exact unclear point and ask the Product Owner, business analyst, lead, or relevant stakeholder a specific question. I will provide an example so the ambiguity is easy to understand. After clarification, I will update the test case or acceptance criteria and confirm that the team shares the same understanding.

## 61. What will you do if a developer rejects your defect?

**Interview answer:** I will first understand the reason for rejection and verify the build, environment, configuration, data, and requirement. I will reproduce the issue and share clear steps, expected and actual results, logs, screenshots, or database evidence. If the expected behavior is unclear, I will request product clarification. My objective is the correct product decision, not proving another person wrong.

## 62. How do you decide what to test first?

**Interview answer:** I use risk-based testing. I prioritize by business impact, customer usage, recent change, technical complexity, integration dependency, defect history, and probability of failure. Critical flows such as login, payment, security, and customer transactions are tested before low-risk cosmetic areas.

## 63. What if you have limited time for testing?

**Interview answer:** I will communicate the available time and scope, perform smoke testing, and focus on critical journeys, changed areas, integrations, and high-risk regression. I will clearly report completed testing, untested areas, unresolved defects, and remaining risk. I will not silently reduce the testing scope.

## 64. Why should we select you for QEA?

**Interview answer:** I have developed a clear foundation in manual testing, SQL, Java, Agile, and Selenium concepts. I can understand requirements, prepare structured tests, validate data, and communicate defects clearly. I am consistent, receptive to feedback, and willing to learn. As a trainee, I know I still have more to learn, but I am ready to take ownership, communicate blockers early, and contribute through disciplined testing while building stronger automation skills.

## 65. What are your strengths?

**Interview answer:** My first strength is structured problem-solving. I understand the requirement, reproduce the issue, verify the environment and data, and collect evidence before reaching a conclusion. My second strength is consistency: I follow defined steps and track work to closure. My third strength is learning ability, and I continuously improve my testing, SQL, Java, Agile, and automation knowledge.

## 66. What is your weakness?

**Interview answer:** Earlier, I sometimes spent more time than necessary perfecting low-risk details. I realized that this could affect speed when several tasks had the same deadline. I now confirm priorities, use business risk to decide the required depth, and time-box lower-priority activities. This helps me maintain quality while completing important work on time.

## 67. Where do you see yourself in three to five years?

**Interview answer:** I want to become a dependable Quality Engineer who can understand business requirements, design effective tests, validate data with SQL, and create maintainable automation. I also want to develop API testing, CI/CD, performance testing, and quality-metric skills. As I gain experience, I would like to take ownership of a module and support newer team members.

### 68. How do you handle pressure and deadlines?

**Interview answer:** I list deliverables, deadlines, dependencies, and risks, then prioritize critical business work and tasks that block others. I estimate effort, communicate capacity early, and provide visible progress. If everything cannot be completed, I explain the trade-off and obtain a priority decision instead of reducing quality silently.

### 69. What will you do if you do not know an interview answer?

**Interview answer:** I will not invent an answer or claim experience I do not have. I will state what I understand, ask a clarifying question if appropriate, explain how I would verify the topic, and offer to discuss the related concept that I know.

**Remember:** Honest reasoning is stronger than a confident but inaccurate answer.

### 70. Do you have any questions for the interviewer?

**Interview answer:** Yes. I can ask: What would success look like for a trainee in the first three months? What testing activities and tools does the team use most often? How does the team balance manual testing, automation, and early quality activities? What learning opportunities help a trainee become productive in the team?

**Remember:** Ask questions that show preparation, curiosity, and interest in contribution.

# Final 30-Minute Revision Checklist

- Explain Java, JDK/JRE/JVM, class, object, constructor, and four OOP pillars.
- Explain new, this, super, extends, implements, interface, abstract, static, final, and access modifiers.
- Write palindrome, factorial, prime, Armstrong, Fibonacci, and reverse-number programs.
- Write CRUD, join, GROUP BY, second-highest salary, duplicate, and NULL queries.
- Explain error, defect, bug, failure, severity, priority, defect lifecycle, SDLC, STLC, smoke, sanity, retesting, and regression.
- Explain Agile roles, artifacts, events, user story, acceptance criteria, and Definition of Done.
- Deliver the self-introduction naturally in 60 to 90 seconds.
- Explain the project approach and why testing starts from requirements.

# Important Note

This is an independent preparation guide designed for an entry-level QEA interview. Actual interim-round questions can vary by trainer, interviewer, account, technology track, and project requirements. Use the answers as speaking frameworks and keep all personal examples truthful.

# ADDITIONAL JAVA, TESTNG AND SELENIUM TOPICS

Supplement to the Cognizant QEA Focused Preparation Guide

Abstract Class and Interface | Collections | Exceptions | Java 8 | Agile | Waits | TestNG | Selenium Basics

# 1. ABSTRACT CLASS AND INTERFACE

## 1. What is an abstract class?

**Interview answer:** An abstract class is a class declared with the **abstract** keyword. It can contain abstract methods without a body, concrete methods with implementation, fields, and constructors. An abstract class cannot be instantiated directly. It is useful when related child classes share common state or behavior.

```
abstract class Vehicle {
    abstract void start();
    void stop() { System.out.println("Stopped"); }
}
class Car extends Vehicle {
    @Override
    void start() { System.out.println("Car started"); }
}
```

**Remember: An abstract class provides a partial implementation and a common base for related classes.**

## 2. What is an interface?

**Interview answer:** An interface defines a contract that implementing classes agree to follow. A class uses the **implements** keyword and provides implementations for the required abstract methods. Interfaces support abstraction and allow a class to implement multiple interface types.

```
interface Payment {
    void pay(double amount);
}
class CardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount);
    }
}
```

**Remember: An interface defines what a class must do; the implementing class defines how it does it.**

## 3. Abstract class versus interface

**Interview answer:** Use an abstract class when closely related classes need shared fields, constructors, or common implementation. Use an interface when different classes should follow the same contract. A class can extend only one class but can implement multiple interfaces.

- Abstract class keyword: abstract class. Interface keyword: interface.
- A child class extends an abstract class. A class implements an interface.
- An abstract class can hold instance state and constructors.
- An interface is mainly used to define capabilities or contracts.
- Java 8 interfaces can also contain default and static methods.

# 2. COLLECTIONS, HASHMAP, LIST, AND SET

## 4. What is the Java Collections Framework?

**Interview answer:** The Java Collections Framework is a standard set of interfaces, implementations, and algorithms for storing and manipulating groups of objects. Important interfaces include Collection, List, Set, Queue, and Deque. Map is part of the framework but is not a subtype of Collection.

- List implementations: ArrayList and LinkedList.
- Set implementations: HashSet, LinkedHashSet, and TreeSet.
- Map implementations: HashMap, LinkedHashMap, and TreeMap.
- The Collections utility class provides algorithms such as sort, reverse, min, and max.

## 5. What is a List?

**Interview answer:** A List stores elements in an ordered sequence, preserves positional access, and allows duplicate values. ArrayList is commonly used when indexed access is important. LinkedList can be useful for operations at the ends or when an iterator performs insertions and removals.

```
List<String> names = new ArrayList<>();
names.add("Lakshay");
names.add("Aman");
names.add("Lakshay");
System.out.println(names.get(0));
```

## 6. What is a Set?

**Interview answer:** A Set stores unique elements. HashSet does not guarantee iteration order. LinkedHashSet preserves insertion order. TreeSet stores elements in sorted order.

```
Set<Integer> ids = new HashSet<>();
ids.add(101);
ids.add(102);
ids.add(101);  // duplicate is not added
```

## 7. What is a HashMap?

**Interview answer:** HashMap stores data as key-value pairs. Keys are unique, while values may be duplicated. HashMap allows one null key and multiple null values, does not guarantee iteration order, and is not synchronized by default.

```
Map<Integer, String> employees = new HashMap<>();
employees.put(101, "Lakshay");
employees.put(102, "Aman");
System.out.println(employees.get(101));
System.out.println(employees.containsKey(102));
```

**Remember: List allows duplicates, Set stores unique values, and Map stores key-value pairs with unique keys.**

# 3. THROW, THROWS, AND STRING CLASSES

## 8. What is throw?

**Interview answer:** The **throw** keyword explicitly throws one exception object from a block or method. It is used when the program detects an invalid condition and wants to transfer control to exception handling code.

```
if (age < 18) {
    throw new IllegalArgumentException("Age must be 18 or above");
}
```

## 9. What is throws?

**Interview answer:** The **throws** keyword appears in a method declaration. It informs the caller that the method may pass one or more exceptions instead of handling them completely inside the method.

```
void readFile() throws IOException {
    Files.readString(Path.of("data.txt"));
}
```

**Remember: throw is used inside code to throw an exception object; throws is used in a method signature to declare possible exceptions.**

## 10. String, StringBuilder, and StringBuffer

**Interview answer:** String is immutable, so its value cannot be changed after creation. StringBuilder is mutable and is preferred for repeated text changes in normal single-threaded code. StringBuffer is mutable and synchronizes many methods, so it can be used when the same buffer is shared across threads, although synchronization adds overhead.

```
StringBuilder builder = new StringBuilder("QEA");
builder.append(" Interview");
System.out.println(builder.toString());

StringBuffer buffer = new StringBuffer("Java");
buffer.append(" Selenium");
```

**Remember: String is immutable; StringBuilder is mutable and usually faster; StringBuffer is mutable and synchronized.**

# 4. JAVA 8 FEATURES

## 11. What important features were introduced in Java 8?

**Interview answer:** Java 8 introduced major language and library improvements that support concise and functional-style programming.

- Lambda expressions: shorter implementations for functional interfaces.
- Functional interfaces: interfaces with one abstract method, such as Predicate, Function, Consumer, and Supplier.
- Method references: concise references to existing methods using ::.
- Stream API: functional-style processing of collections using operations such as filter, map, and reduce.
- Default and static methods in interfaces.
- Optional: a container used to represent a value that may be absent.
- New Date and Time API in java.time.
- forEach and other collection-related enhancements.

```
List<String> names = Arrays.asList("Lakshay", "Aman", "Ravi");
names.stream()
     .filter(name -> name.startsWith("L"))
     .forEach(System.out::println);
```

**Remember: For an interview, focus first on lambda expressions, functional interfaces, Stream API, method references, Optional, and the Date-Time API.**

## 12. What is a lambda expression?

**Interview answer:** A lambda expression is a compact way to provide the implementation of a functional interface. It reduces boilerplate code and is commonly used with collections and streams.

```
List<Integer> numbers = Arrays.asList(3, 1, 2);
numbers.forEach(number -> System.out.println(number));
```

## 13. What is the Stream API?

**Interview answer:** The Stream API processes data through a pipeline of operations without changing the source collection. Intermediate operations include filter and map. Terminal operations include collect, count, reduce, and forEach.

```
List<Integer> even = numbers.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

# 5. AGILE METHODOLOGY

## 14. What is Agile methodology?

**Interview answer:** Agile is an iterative and incremental development approach in which cross-functional teams deliver usable value in short cycles, obtain feedback, and adapt to change. Testing happens continuously rather than only after all development is complete.

- Product Backlog: ordered list of product work.
- Sprint: fixed-length cycle used to work toward a Sprint Goal.
- User story: short description of desired value from a user perspective.
- Acceptance criteria: testable conditions for accepting the story.
- Daily Scrum: short event for Developers to inspect progress and adapt their plan.
- Sprint Review: inspection of the Increment with stakeholders.
- Retrospective: review of how the team worked and what should improve.
- Definition of Done: shared quality standard for completed work.

**Remember: In Agile, quality is a continuous team responsibility throughout each Sprint.**

## 15. What is the tester’s role in Agile?

**Interview answer:** The tester reviews stories and acceptance criteria, identifies risks, prepares tests and data, tests continuously during the Sprint, reports defects, supports automation, retests fixes, runs regression, and communicates quality status. The tester collaborates with the Product Owner, developers, and other team members from refinement through review and retrospective.

# 6. IMPLICIT AND EXPLICIT WAITS

## 16. What is an implicit wait?

**Interview answer:** An implicit wait sets a global timeout for element-finding operations in a WebDriver session. When an element is not immediately available, WebDriver polls until the element is found or the timeout expires.

```
driver.manage().timeouts()
      .implicitlyWait(Duration.ofSeconds(10));
```

## 17. What is an explicit wait?

**Interview answer:** An explicit wait waits for a specific condition for a specific element or situation, such as visibility, presence, clickability, title, or URL. Explicit waits are generally more precise for dynamic applications.

```
WebDriverWait wait = new WebDriverWait(
    driver, Duration.ofSeconds(10));
WebElement login = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("login")));
login.click();
```

## 18. Implicit wait versus explicit wait

**Interview answer:** Implicit wait is global and mainly affects element lookup. Explicit wait is local and waits for a defined condition. Fixed Thread.sleep pauses execution for a set time and should normally be avoided because it is slow and does not react to application readiness.

**Remember: Prefer condition-based explicit waits for dynamic elements. Avoid mixing wait strategies without understanding the timing effect.**

# 7. TESTNG: BEFORECLASS AND BEFOREMETHOD

## 19. What is @BeforeClass?

**Interview answer:** In TestNG, a method annotated with **@BeforeClass** runs once before the first test method in the current class. It is useful for setup shared by all test methods in that class, such as creating a driver or loading class-level configuration.

## 20. What is @BeforeMethod?

**Interview answer:** A method annotated with **@BeforeMethod** runs before each @Test method. It is useful when every test needs a fresh page state, test data, navigation step, or other repeatable setup.

```
public class LoginTests {
    WebDriver driver;

    @BeforeClass
    public void startBrowser() {
        driver = new ChromeDriver();
    }

    @BeforeMethod
    public void openLoginPage() {
        driver.get("https://example.com/login");
    }

    @Test
    public void validLogin() { }

    @Test
    public void invalidLogin() { }
}
```

**Remember: @BeforeClass runs once per class; @BeforeMethod runs before every test method.**

# 8. SELENIUM BASICS

## 21. What is Selenium?

**Interview answer:** Selenium is an open-source project for automating web browsers. It is commonly used to automate functional, regression, and cross-browser tests for web applications. Selenium does not directly automate native desktop applications.

## 22. What is WebDriver?

**Interview answer:** WebDriver is Selenium's main browser automation API. It communicates with a browser-specific driver or remote Selenium server and performs browser actions in a way similar to a user.

```
WebDriver driver = new ChromeDriver();
driver.get("https://example.com");
System.out.println(driver.getTitle());
driver.quit();
```

## 23. What is Selenium Grid?

**Interview answer:** Selenium Grid supports remote and parallel test execution across different browsers, browser versions, and operating environments. It reduces execution time and helps increase browser coverage.

## 24. What is a WebElement?

**Interview answer:** WebElement represents an HTML element in the Document Object Model. After locating a WebElement, a test can click it, type text, read its value or text, check visibility, or inspect attributes.

```
WebElement username = driver.findElement(By.id("username"));
username.sendKeys("lakshay");
System.out.println(username.isDisplayed());
```

## 25. What are Selenium locators?

**Interview answer:** Locators identify elements in the page DOM. Selenium supports id, name, class name, tag name, link text, partial link text, CSS selector, and XPath.

```
driver.findElement(By.id("username"));
driver.findElement(By.name("password"));
driver.findElement(By.cssSelector("button[type='submit']"));
driver.findElement(By.xpath("//input[@id='username']"));
```

**Remember: Choose a locator that is unique, stable, and readable. Prefer stable identifiers or dedicated test attributes when available.**

# 9. XPATH, ABSOLUTE XPATH, AND RELATIVE XPATH

## 26. What is XPath?

**Interview answer:** XPath is a query language used to locate nodes in XML or HTML documents. In Selenium, XPath can locate elements using tag names, attributes, visible text, hierarchy, conditions, and relationships between elements.

## 27. What is absolute XPath?

**Interview answer:** Absolute XPath starts from the root html element and describes the complete path to the target. It usually begins with a single slash. Absolute XPath is fragile because a small structural change can break the complete path.

```
/html/body/div[1]/form/input[1]
```

## 28. What is relative XPath?

**Interview answer:** Relative XPath can start from anywhere in the document and commonly begins with two slashes. It can use stable attributes, text, functions, and relationships. Relative XPath is generally preferred because it is shorter and less dependent on the complete page structure.

```
//input[@id="username"]
//button[normalize-space()="Login"]
//input[contains(@name,"user")]
//label[normalize-space()="Email"]/following::input[1]
```

## 29. Common XPath syntax

**Interview answer:** Use @ for an attribute, square brackets for a condition, contains() for partial matching, starts-with() for prefixes, normalize-space() for cleaned text, and axes for element relationships.

- //input[@name="email"] selects an input by attribute.
- //button[contains(@class,"submit")] selects a button with a partial class value.
- //div[@id="form"]/child::input selects child input elements.
- //label[text()="Password"]/following-sibling::input selects a related sibling.

**Remember: Prefer a stable relative XPath over a long absolute XPath.**

# Final Revision

Be ready to explain every topic in 30 to 60 seconds and write the short Java, TestNG, and Selenium examples without copying. This supplement is an independent learning resource; actual interview coverage can vary by project and interviewer.

# ADVANCED QEA INTERVIEW TOPICS

Selenium Framework | TestNG | Data-Driven Testing | Maven | SQL | Manual Testing | Agile | Communication

Interview-ready explanations and practical code examples

# 1. JAVA AND SELENIUM DESIGN CONCEPTS

## 1. Differentiate an abstract class and an interface

**Interview answer:** An abstract class is a partially implemented base class for closely related classes. An interface defines a capability or contract that different classes can implement.

- A class uses extends for an abstract class and implements for an interface.
- A class can directly extend only one class, but it can implement multiple interfaces.
- An abstract class can have constructors, instance fields, abstract methods, and concrete methods.
- An interface cannot be instantiated and is mainly used to specify behavior. Java 8 interfaces can contain default and static methods.
- Use an abstract class for shared state or common base implementation. Use an interface for a common contract across potentially unrelated classes.

```
abstract class BasePage {
    WebDriver driver;
    BasePage(WebDriver driver) { this.driver = driver; }
    abstract boolean isLoaded();
}

interface ScreenshotCapable {
    void captureScreenshot();
}
```

**Remember: Abstract class means shared base plus partial implementation; interface means a contract or capability.**

## 2. What are locators in Selenium?

**Interview answer:** Locators are strategies used by WebDriver to identify elements in the page DOM. Selenium supports id, name, class name, tag name, link text, partial link text, CSS selector, and XPath.

- Prefer a unique and stable id or a dedicated test attribute when available.
- Use CSS or a stable relative XPath when simple attributes are not enough.
- Avoid absolute XPath and dynamic indexes because small DOM changes can break them.
- Keep locators inside page objects instead of repeating them in test methods.

```
By username = By.id("username");
By submit = By.cssSelector("button[type='submit']");
By message = By.xpath("//div[@role='alert']");
```

## 3. Explain Selenium WebDriver architecture

**Interview answer:** A test is written using Selenium language bindings such as Java. The WebDriver API sends standardized commands to a local browser driver or a remote Selenium server. The browser-side implementation performs the requested action and returns a response containing success, data, or an error.

- Test script: test logic written in Java.
- Selenium client library: provides WebDriver, WebElement, By, waits, and support APIs.
- W3C WebDriver protocol: standard command and response communication.
- Browser driver or remote end: ChromeDriver, GeckoDriver, EdgeDriver, or Selenium Grid node.
- Browser: executes navigation and element interactions, then returns the result.
- Selenium Grid adds routing and distributed execution across machines and browsers.

```
TestNG test -> Selenium Java API -> W3C WebDriver commands
            -> browser driver / Grid node -> browser
            <- response or WebDriver exception <-
```

**Remember: In Selenium 4, WebDriver communication follows the W3C WebDriver standard.**

## 4. What is an implicit wait?

**Interview answer:** Implicit wait is a global timeout used by the WebDriver session for element-location calls. If an element is not immediately found, WebDriver polls until the element is found or the timeout ends.

```
driver.manage().timeouts()
      .implicitlyWait(Duration.ofSeconds(10));
```

**Remember: Implicit wait is global. For dynamic states such as clickability or visibility, use a condition-based explicit wait. Do not mix wait types without understanding the timing effect.**

# 2. PAGE OBJECT MODEL, CLICKS, AND ACTIONS

## 5. What is the Page Object Model?

**Interview answer:** Page Object Model, or POM, is a design pattern where each application page or major component is represented by a class. Locators and page-level actions are kept inside that class, while tests express business flows.

- Improves readability by hiding low-level locator code.
- Reduces duplication because locators are maintained in one place.
- Makes UI changes easier to update.
- Page objects should expose meaningful actions such as login(), not every technical operation.
- Assertions normally remain in the test or assertion layer, except page-state checks used for synchronization.

```
class LoginPage {
    private WebDriver driver;
    private By user = By.id("username");
    private By password = By.id("password");
    private By login = By.id("login");

    LoginPage(WebDriver driver) { this.driver = driver; }

    HomePage loginAs(String u, String p) {
        driver.findElement(user).sendKeys(u);
        driver.findElement(password).sendKeys(p);
        driver.findElement(login).click();
        return new HomePage(driver);
    }
}
```

## 6. What are different ways to click in Selenium?

**Interview answer:** Use the normal WebElement click first. Other click approaches are used only when the user interaction requires them or when the application has special behavior.

- WebElement.click(): standard and preferred click.
- Actions.click(element): mouse-based click through the Actions API.
- Actions.doubleClick(element): double-click.
- Actions.contextClick(element): right-click.
- Actions.moveToElement(element).click(): hover then click.
- JavaScript click: arguments[0].click(); use carefully because it can bypass normal interactability checks and may not represent a real user action.
- Keyboard activation: sendKeys(Keys.ENTER) or SPACE when appropriate for the control.

```
element.click();

new Actions(driver).moveToElement(element).click().perform();
new Actions(driver).doubleClick(element).perform();
new Actions(driver).contextClick(element).perform();

((JavascriptExecutor) driver)
    .executeScript("arguments[0].click();", element);
```

**Remember: Prefer the normal click. Use an explicit wait and investigate overlays or state problems before using JavaScript click.**

## 7. What is the Actions class used for?

**Interview answer:** The Selenium Actions class builds complex keyboard and mouse interactions. It is used for hover, drag and drop, click-and-hold, right-click, double-click, key combinations, and chained user gestures.

```
Actions actions = new Actions(driver);
actions.moveToElement(menu)
       .click(subMenu)
       .perform();
```

```
actions.keyDown(Keys.CONTROL)
       .sendKeys("a")
       .keyUp(Keys.CONTROL)
       .perform();
```

## 8. driver.close() versus driver.quit()

**Interview answer:** driver.close() closes the current browser window or tab controlled by the active context. driver.quit() ends the complete WebDriver session and closes every associated browser window.

**Remember: Use quit() during final cleanup so the complete session and driver resources are released.**

# 3. TESTNG ANNOTATIONS AND EXECUTION FLOW

## 9. What are TestNG annotations?

**Interview answer:** TestNG annotations control test execution, setup, teardown, grouping, parameters, and data-driven behavior.

- @Test marks a test method.
- @BeforeSuite and @AfterSuite run around the complete suite.
- @BeforeTest and @AfterTest run around the classes inside a test tag in testng.xml.
- @BeforeClass and @AfterClass run around the test methods in one class.
- @BeforeMethod and @AfterMethod run before and after every test method.
- @BeforeGroups and @AfterGroups run around specified groups.
- @DataProvider supplies multiple datasets.
- @Parameters reads parameters commonly supplied from testng.xml.
- @Factory creates multiple test-class instances.
- @Listeners attaches listener implementations.

## 10. Explain before and after annotations

**Interview answer:** Before annotations prepare preconditions. After annotations perform cleanup, evidence capture, and resource release. The level determines how often they run.

- @BeforeSuite -> before everything in the suite.
- @BeforeTest -> before classes in one XML test block.
- @BeforeClass -> once before the first test method in a class.
- @BeforeMethod -> before each @Test method.
- @AfterMethod -> after each @Test method.
- @AfterClass -> once after all methods in the class.
- @AfterTest -> after the XML test block.
- @AfterSuite -> after the complete suite.

```
@BeforeClass
public void createDriver() { driver = new ChromeDriver(); }

@BeforeMethod
public void openApplication() { driver.get(baseUrl); }

@AfterMethod
public void collectResult(ITestResult result) {
    if (!result.isSuccess()) captureScreenshot();
}

@AfterClass(alwaysRun = true)
public void stopDriver() { if (driver != null) driver.quit(); }
```

**Remember: BeforeClass runs once per class; BeforeMethod runs before every test method. The matching After annotations perform cleanup at the same level.**

# 4. ROBUST ELEMENT HANDLING AND TEST DATA

## 11. How do you handle NoSuchElementException?

**Interview answer:** NoSuchElementException means WebDriver could not locate an element in the current search context. Do not solve it by blindly adding sleep. Identify the actual reason and synchronize with the application.

- Verify the locator in the current DOM and check whether it is unique.
- Confirm that the correct page, window, frame, and browser context are active.
- Switch into an iframe before finding elements inside it.
- Wait explicitly for presence, visibility, or clickability as required.
- Check whether the element is inside Shadow DOM.
- Confirm that the element has not been replaced by a re-render. Re-locate it instead of holding an old reference.
- Capture screenshot, page URL, page source, and logs for diagnosis.
- Use findElements() when absence is an expected condition because it returns an empty list instead of throwing NoSuchElementException.

```
WebDriverWait wait = new WebDriverWait(
    driver, Duration.ofSeconds(10));

WebElement element = wait.until(
    ExpectedConditions.visibilityOfElementLocated(
        By.id("message")));

List<WebElement> banners = driver.findElements(By.id("optionalBanner"));
if (!banners.isEmpty()) { banners.get(0).click(); }
```

## 12. How do you read an Excel file in Java?

**Interview answer:** In Java automation, Apache POI is commonly used to read .xls and .xlsx workbooks. For .xlsx, use XSSFWorkbook or the common WorkbookFactory API. Open the file with try-with-resources, select the sheet, iterate rows and cells, and convert values safely.

```
try (FileInputStream input =
         new FileInputStream("src/test/resources/testdata.xlsx");
     Workbook workbook = WorkbookFactory.create(input)) {

    Sheet sheet = workbook.getSheet("LoginData");
    DataFormatter formatter = new DataFormatter();

    for (Row row : sheet) {
        String username = formatter.formatCellValue(row.getCell(0));
        String password = formatter.formatCellValue(row.getCell(1));
        System.out.println(username + " " + password);
    }
}
```

**Remember: Add Apache POI poi-ooxml as a Maven dependency for .xlsx files and close workbook resources properly.**

## 13. How do you externalize test data instead of hardcoding it?

**Interview answer:** Keep test data and environment configuration outside test methods. Choose the source based on the type of data and maintenance needs.

- properties or YAML: URLs, browser name, timeouts, and environment values.
- TestNG @DataProvider: multiple datasets supplied to tests.
- CSV or Excel: tabular business data used by non-developers.
- JSON: nested request or domain data.
- Environment variables or secret stores: passwords, tokens, and confidential values.
- Database or API: dynamically managed shared test data.

- Command-line or Maven system properties: runtime environment selection.

```
Properties config = new Properties();
try (FileInputStream in =
         new FileInputStream("src/test/resources/config.properties")) {
    config.load(in);
}
String baseUrl = config.getProperty("baseUrl");

@DataProvider(name = "loginData")
public Object[][] loginData() {
    return new Object[][] {
        {"validUser", "validPassword"},
        {"lockedUser", "password"}
    };
}
```

**Remember: Never store real credentials in source code or normal test-data files. Use environment variables or an approved secret manager.**

# 5. MAVEN AUTOMATION PROJECT

## 14. What is a Maven project and why is Maven used in automation?

**Interview answer:** Maven is a Java build and dependency-management tool. A Maven project is described by pom.xml, which defines coordinates, dependencies, plugins, properties, and build behavior.

- Downloads and manages Selenium, TestNG, Apache POI, logging, and reporting dependencies.
- Provides a standard folder structure: src/main/java, src/test/java, and src/test/resources.
- Runs repeatable lifecycle commands such as clean, compile, test, and package.
- Uses Surefire to execute unit or TestNG tests during the test phase.
- Supports profiles and system properties for different environments.
- Integrates consistently with IDEs and CI/CD tools.

```
mvn clean test
mvn test -Dbrowser=chrome -Denv=qa

<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>${selenium.version}</version>
  </dependency>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>${testng.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

**Remember: Maven makes an automation project reproducible by standardizing dependencies, builds, and test execution.**

# 6. SQL: SELECT, JOINS, WHERE, AND HAVING

## 15. Explain all common SQL joins

**Interview answer:** A join combines rows from two table sources using a relationship condition.

- INNER JOIN: only rows that match in both tables.
- LEFT JOIN: all left-table rows plus matching right-table data; unmatched right columns are NULL.
- RIGHT JOIN: all right-table rows plus matching left-table data.
- FULL OUTER JOIN: matched rows and unmatched rows from both tables.
- CROSS JOIN: every left row paired with every right row.
- SELF JOIN: a table joined to itself, commonly for employee-manager relationships.

## 16. Query for SELECT

**Interview answer:** SELECT retrieves columns and rows from a table. Use explicit column names for readable and stable queries.

```
SELECT employee_id, employee_name, salary
FROM Employee
WHERE status = 'ACTIVE'
ORDER BY salary DESC;
```

## 17. Query for INNER JOIN

**Interview answer:** This query returns employees whose department_id matches a Department row.

```
SELECT e.employee_id,
       e.employee_name,
       d.department_name
FROM Employee e
INNER JOIN Department d
    ON e.department_id = d.department_id;
```

## 18. JOIN query examples

**Interview answer:** Each query changes which unmatched rows are retained.

```
-- LEFT JOIN
SELECT e.employee_name, d.department_name
FROM Employee e
LEFT JOIN Department d
  ON e.department_id = d.department_id;

-- FULL OUTER JOIN
SELECT e.employee_name, d.department_name
FROM Employee e
FULL OUTER JOIN Department d
  ON e.department_id = d.department_id;

-- CROSS JOIN
SELECT e.employee_name, s.shift_name
FROM Employee e CROSS JOIN Shift s;

-- SELF JOIN
SELECT e.employee_name, m.employee_name AS manager_name
FROM Employee e
LEFT JOIN Employee m ON e.manager_id = m.employee_id;
```

## 19. WHERE versus HAVING

**Interview answer:** WHERE filters individual rows before grouping and aggregate calculation. HAVING filters groups after GROUP BY. Use WHERE for normal row conditions and HAVING for conditions involving aggregate results.

```
SELECT department_id, COUNT(*) AS employee_count
FROM Employee
WHERE status = 'ACTIVE'
GROUP BY department_id
HAVING COUNT(*) > 5;
```

**Remember: WHERE filters rows; HAVING filters grouped results.**

# 7. STLC, DEFECT LIFE CYCLE, AND EXPLORATORY TESTING

## 20. Explain STLC

**Interview answer:** STLC means Software Testing Life Cycle. It organizes testing activities from requirement understanding through closure.

- Requirement analysis: identify testable requirements, questions, risks, and traceability.
- Test planning: define scope, approach, effort, resources, tools, schedule, entry/exit criteria, and risks.
- Test design: prepare scenarios, test cases, automation candidates, and test data.
- Environment setup: confirm application build, services, integrations, access, and data.
- Test execution: run tests, record evidence, compare actual and expected results.
- Defect management: report, track, retest, and communicate defects.
- Regression and reporting: assess change impact and provide quality status.
- Test closure: summarize coverage, open risk, lessons, metrics, and sign-off information.

**Remember: STLC converts requirements into planned coverage, execution evidence, defect information, and an informed closure decision.**

## 21. Explain the defect or bug life cycle

**Interview answer:** A typical defect flow is New, Assigned, Open or In Progress, Fixed, Retest, and Closed. If the issue remains, the tester reopens it.

- New: tester logs the defect with evidence.
- Assigned: ownership is given to the responsible person or team.
- Open/In Progress: defect is analyzed and corrected.
- Fixed: developer provides the build containing the change.
- Retest: tester verifies the exact failed condition.
- Closed: expected behavior is verified.
- Reopened: fix failed or the same issue remains.
- Other states can include Duplicate, Rejected, Deferred, Cannot Reproduce, and Not a Defect.

**Remember: Retest verifies the fix; regression checks related functionality for side effects.**

## 22. What is exploratory testing?

**Interview answer:** Exploratory testing is an approach in which learning, test design, and execution happen together. The tester uses product knowledge, risk, observation, and previous results to guide the next test.

- Define a test charter containing scope, objective, and risks.
- Use a time-boxed session.
- Record test data, paths, observations, questions, and defects.
- Combine exploration with structured techniques such as boundaries, states, and error guessing.
- Debrief the session and convert important discoveries into repeatable tests when needed.

**Remember: Exploratory testing is structured investigation, not random clicking.**

# 8. AGILE, SPRINTS, AND SCRUM EVENTS

## 23. What is Agile methodology? Explain Sprint and ceremonies

**Interview answer:** Agile is an iterative and incremental approach that delivers usable value in short cycles and adapts through feedback. Scrum is a commonly used Agile framework. A Sprint is a fixed-length event of one month or less in which the team works toward a Sprint Goal and creates a usable Increment.

- Backlog Refinement: ongoing clarification and breakdown of future Product Backlog items. It is commonly practised but is not one of Scrum's five formal events.
- Sprint Planning: decide why the Sprint is valuable, what work is selected, and how it will be completed.
- Daily Scrum: a 15-minute event for Developers to inspect progress toward the Sprint Goal and adapt the plan.
- Sprint Review: inspect the Increment with stakeholders and discuss what to do next.
- Sprint Retrospective: inspect team effectiveness and identify improvements.
- The Sprint itself is also a Scrum event and contains the other events.

**Remember: Scrum formally defines five events: Sprint, Sprint Planning, Daily Scrum, Sprint Review, and Sprint Retrospective.**

# 9. PROJECT AND COMMUNICATION ANSWERS

## 24. Walk me through exception handling in your project

**Interview answer:** Use an honest framework answer if you have training or practice exposure rather than production experience. Explain where exceptions are handled and what evidence is captured.

- Use try-with-resources for files, workbooks, database connections, or streams.
- Catch specific exceptions where meaningful recovery or reporting is possible.
- Let unexpected automation failures reach TestNG so the test is marked failed.
- Use @AfterMethod or a listener to capture screenshots, URL, test name, and logs after failure.
- Use explicit waits for expected timing conditions instead of catching NoSuchElementException repeatedly.
- Create framework-specific exceptions only when they add useful context.
- Log useful information without exposing passwords, tokens, or sensitive test data.
- Always close WebDriver and external resources in controlled cleanup.

```
@AfterMethod(alwaysRun = true)
public void afterMethod(ITestResult result) {
    try {
        if (!result.isSuccess() && driver != null) {
            captureScreenshot(result.getName());
        }
    } catch (RuntimeException evidenceError) {
        logger.error("Evidence capture failed", evidenceError);
    } finally {
        // Per-method cleanup if required
    }
}

@AfterClass(alwaysRun = true)
public void tearDown() {
    if (driver != null) driver.quit();
}
```

**Remember: Do not say every exception is caught. Catch only where you can recover, add context, or perform controlled cleanup.**

## 25. Explain a technical concept to a non-technical person

**Interview answer:** Begin with the purpose, use a familiar analogy, explain the outcome, and avoid unnecessary jargon. Then check whether the listener wants more detail.

- Example using Selenium WebDriver: “WebDriver is like a remote control for a browser. The automation script asks the browser to open a page, enter information, and select buttons. After each step, the browser reports what happened, allowing the test to compare the result with what was expected.”
- Example using API: “An API is like a restaurant waiter. A customer gives an order to the waiter, the waiter carries it to the kitchen, and the waiter returns with the result. The customer does not need to know how the kitchen prepares it.”
- Structure: goal -> analogy -> simple process -> business impact -> confirm understanding.

**Remember: Simple explanation does not mean inaccurate explanation. Preserve the main idea and remove unnecessary implementation detail.**

# Final Practice Checklist

- Draw the WebDriver architecture and explain each component.
- Write a Page Object, explicit wait, TestNG lifecycle, Excel reader, and Maven command.
- Explain every join and write SELECT, INNER JOIN, and WHERE/HAVING queries.
- Speak STLC, defect lifecycle, exploratory testing, Sprint events, and project exception handling aloud.
- Practise the non-technical explanations in under one minute.

# SELENIUM FOLLOW-UP INTERVIEW TOPICS

XPath | Locators | Driver Setup | Frames | Windows | Pagination | Mouse Actions | Framework Usage

# 1. RELATIVE XPATH AND LOCATORS

## 1. What is relative XPath?

**Interview answer:** Relative XPath locates an element from anywhere in the DOM instead of starting from the root html node. It normally begins with two forward slashes and uses stable attributes, text, functions, or element relationships.

- Use @ to select an attribute.
- Use contains() for a stable partial value.
- Use normalize-space() to remove extra spaces from visible text.
- Use axes such as parent, child, ancestor, following, and following-sibling for relationships.
- Avoid long index-based paths when a stable attribute is available.

```
//input[@id="username"]
//button[normalize-space()="Search"]
//input[contains(@name,"source")]
//label[normalize-space()="Email"]/following-sibling::input
//div[@id="result"]/ancestor::section
```

**Remember: Relative XPath is generally more maintainable than absolute XPath, but the best locator is the most stable and unique locator.**

## 2. What are locators?

**Interview answer:** Locators are strategies that identify elements in the page DOM so WebDriver can interact with them.

- id: locates an element by its id attribute.
- name: locates by the name attribute.
- className: locates by one class value, not a compound class string.
- tagName: locates by HTML tag.
- linkText and partialLinkText: locate anchor elements using visible text.
- cssSelector: locates using CSS syntax.
- xpath: locates using XPath expressions.

```
driver.findElement(By.id("username"));
driver.findElement(By.name("password"));
driver.findElement(By.cssSelector("button[data-testid='login']"));
driver.findElement(By.xpath("//button[normalize-space()='Login']"));
```

**Remember: Preferred order depends on the application, but stable id or dedicated test attributes are usually strong choices.**

## 3. What Selenium version are you using?

**Interview answer:** Answer with the version declared in your project pom.xml, not a guessed latest version. For this preparation guide, the current official stable Java binding is Selenium 4.49.0, released on September 9, 2026. Always verify the project dependency before an interview.

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.49.0</version>
</dependency>
```

**Remember: A good project answer is: “I use the Selenium version defined in pom.xml. I verify it from the selenium-java dependency.”**

# 2. WEBDRIVER INITIALIZATION AND PROJECT EXCEPTIONS

## 4. How do you initialize WebDriver?

**Interview answer:** Declare the reference using the WebDriver interface and create the required browser implementation. In current Selenium versions, Selenium Manager can resolve supported browser drivers automatically in normal setups.

```
WebDriver driver = new ChromeDriver();
driver.manage().window().maximize();
driver.manage().timeouts()
      .implicitlyWait(Duration.ofSeconds(5));
driver.get("https://example.com");

// At the end
driver.quit();
```

## 5. How do you initialize different browsers in a framework?

**Interview answer:** Use a driver factory that receives the browser name from configuration, command-line parameters, TestNG XML, or external test data. Return a WebDriver reference so tests are browser-independent.

```
public static WebDriver createDriver(String browser) {
    switch (browser.toLowerCase()) {
        case "chrome": return new ChromeDriver();
        case "firefox": return new FirefoxDriver();
        case "edge": return new EdgeDriver();
        default: throw new IllegalArgumentException(
            "Unsupported browser: " + browser);
    }
}
```

## 6. Explain an exception you faced during training

**Interview answer:** Use a real example. A suitable answer is: “During training, I faced NoSuchElementException because the locator was correct but the element loaded dynamically. I verified the DOM and added an explicit wait for visibility instead of using Thread.sleep. I also faced a NullPointerException when the Excel sheet name did not match, so I validated the workbook and sheet before reading rows. These issues taught me to verify context, data, and synchronization before changing locators.”

**Remember: Explain the situation, root cause, correction, and lesson. Do not invent production experience.**

## 7. What is ElementClickInterceptedException?

**Interview answer:** This exception occurs when WebDriver finds the target element, but another element receives the click, such as a loader, popup, sticky header, cookie banner, or animation overlay.

- Wait for the target to be clickable.
- Wait for the overlay or loader to disappear.
- Scroll the target into the viewport.
- Close a popup or banner if it blocks the element.
- Verify that the locator points to the visible intended element.
- Use Actions click when a real mouse move is required.
- Use JavaScript click only after understanding why the normal user-like click fails.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.invisibilityOfElementLocated(
    By.cssSelector(".loading-overlay")));
WebElement button = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("submit")));
button.click();
```

**Remember: This is likely what “element interrupt error” refers to. The standard Selenium exception name is ElementClickInterceptedException.**

# 3. IFRAMES AND WINDOW HANDLES

## 8. What is an iframe and how do you handle it?

**Interview answer:** An iframe embeds a separate HTML document inside the current page. WebDriver cannot directly locate elements inside it until the driver switches into that frame.

- Switch by index when the position is stable, though indexes can be fragile.
- Switch by frame name or id.
- Switch using a located WebElement.
- Use parentFrame() to move one level up.
- Use defaultContent() to return to the main document.

```
WebElement frame = driver.findElement(By.id("paymentFrame"));
driver.switchTo().frame(frame);
driver.findElement(By.id("cardNumber")).sendKeys("4111111111111111");
driver.switchTo().defaultContent();

// Explicit wait approach
wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(
    By.id("paymentFrame")));
```

## 9. What is a window handle?

**Interview answer:** A window handle is a unique String identifier for one browser window or tab in the current WebDriver session. getWindowHandle() returns the current handle. getWindowHandles() returns all open handles as a Set.

```
String parent = driver.getWindowHandle();
Set<String> allWindows = driver.getWindowHandles();

for (String window : allWindows) {
    if (!window.equals(parent)) {
        driver.switchTo().window(window);
        break;
    }
}

driver.close();
driver.switchTo().window(parent);
```

## 10. Different ways to obtain and collect window handles

**Interview answer:** The direct return type is Set because handles are unique and order is not guaranteed. Convert the set when indexed access or stream operations are required.

```
// Direct set
Set<String> handles = driver.getWindowHandles();

// Convert to List for indexed access
List<String> windowList = new ArrayList<>(handles);
driver.switchTo().window(windowList.get(1));

// Stream collection
List<String> collected = driver.getWindowHandles()
        .stream()
        .collect(Collectors.toList());

// Wait until expected number of windows exists
wait.until(ExpectedConditions.numberOfWindowsToBe(2));
```

**Remember: Do not assume the first or second Set item is always the parent or child. Save the parent handle before opening the new window and compare handles.**

## 11. Difference between Set and List

**Interview answer:** A List is ordered, supports index-based access, and allows duplicates. A Set stores unique values and generally does not provide index-based access. ArrayList preserves insertion order; HashSet does not guarantee order; LinkedHashSet preserves insertion order; TreeSet sorts values.

**Remember: getWindowHandles() returns Set because every open window handle must be unique.**

# 4. PAGINATION AND MOUSE ACTIONS

## 12. How do you automate pagination?

**Interview answer:** Pagination testing confirms navigation across result pages and verifies that records, page indicators, and controls behave correctly. Avoid hardcoding the number of pages unless the requirement guarantees it.

- Validate Next, Previous, first-page, and last-page behavior.
- Check disabled states on the first and last page.
- Verify that page number and records change after navigation.
- Check for duplicate or missing records across pages if required.
- Wait for the old result table to become stale or for a loading indicator to disappear.
- Use a safety limit to prevent an infinite loop.

```
Set<String> seenItems = new HashSet<>();
int safetyLimit = 100;

for (int pageNo = 1; pageNo <= safetyLimit; pageNo++) {
    List<WebElement> rows = driver.findElements(
        By.cssSelector("table tbody tr"));
    for (WebElement row : rows) {
        seenItems.add(row.getText());
    }

    List<WebElement> next = driver.findElements(By.id("next"));
    if (next.isEmpty() || !next.get(0).isEnabled()) break;

    WebElement oldFirstRow = rows.get(0);
    next.get(0).click();
    wait.until(ExpectedConditions.stalenessOf(oldFirstRow));
}
```

## 13. What is context click?

**Interview answer:** Context click means right-clicking an element. Selenium performs it using the Actions class contextClick() method. This is likely the intended meaning of “Contest Click.”

```
WebElement file = driver.findElement(By.id("fileItem"));
new Actions(driver)
    .contextClick(file)
    .perform();
```

## 14. How do you perform mouse hover?

**Interview answer:** Use Actions.moveToElement() to move the virtual mouse over an element. Then wait for the submenu or tooltip before interacting with it.

```
WebElement menu = driver.findElement(By.id("products"));
new Actions(driver).moveToElement(menu).perform();

WebElement subMenu = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("automation")));
subMenu.click();
```

# 5. FRAMEWORK USAGE IN AUTOMATION

## 15. Do you create a framework or use an existing framework?

**Interview answer:** In a real project, the usual approach is to understand and extend the existing automation framework because it already follows project conventions, reporting, CI execution, utilities, and maintenance standards. A new framework should be created only when the project has no suitable framework or the existing design cannot meet justified requirements.

- First understand the existing folder structure, base classes, driver factory, page objects, utilities, configuration, test-data flow, reporting, listeners, and CI pipeline.
- Follow the current coding and naming conventions.
- Reuse existing utilities instead of duplicating code.
- Add new components in a backward-compatible way.
- Discuss architectural changes with the lead and document the reason.
- For a training or greenfield project, create a small framework using Maven, TestNG, Page Object Model, DriverFactory, waits, configuration, screenshots, reports, logging, and externalized test data.

**Remember: The likely intended term is “framework,” not “firmware.” A strong answer is: “I first evaluate and use the existing framework; I extend it where needed. I build a new framework only when there is a justified project requirement.”**

## 16. Simple framework structure

**Interview answer:** A maintainable beginner framework separates tests, pages, driver creation, utilities, configuration, and test data.

```
src/test/java
  tests/
    FlightSearchTest.java
  pages/
    HomePage.java
    ResultPage.java
  utils/
    DriverFactory.java
    DataUtils.java
    ScreenshotUtil.java
  listeners/
    TestListener.java

src/test/resources
  config.properties
  IRCTC_Flight_Booking_Automation.xlsx
  testng.xml

pom.xml
```

# Final Interview Recall

- Relative XPath uses // and stable conditions.
- getWindowHandles() returns Set; convert to List only when needed.
- Switch into an iframe before locating elements inside it.
- ElementClickInterceptedException usually means another element blocked the click.
- Use contextClick() for right-click and moveToElement() for hover.
- Use or extend the existing framework unless a new framework is justified.